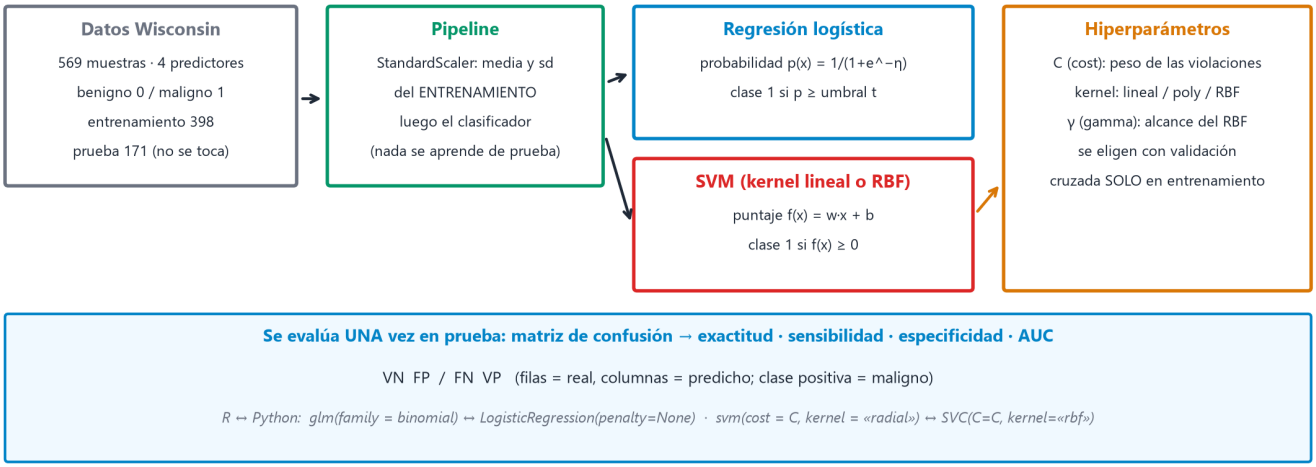
Clase 37 · SVM y clasificadores clásicos

Profesor Pedro Luiz Ramos · regresión logística y máquinas de vectores de soporte · sigue a la Clase 36 (Boosting) y precede a la Clase 38

La gran idea de la clase

Clasificar es construir una regla que, a partir de las variables **X**, asigne una clase (0 o 1). La **regresión logística** estima una **probabilidad** y decide con un **umbral**; la **SVM** busca el **hiperplano de margen máximo**, entrega un **puntaje** f(x) y decide con su signo. El **kernel** permite fronteras curvas y los **hiperparámetros** (C y gamma) se eligen con validación cruzada **solo en entrenamiento**. «No hay un modelo que siempre gane»: se comparan en datos que no se usaron para ajustar.

La clase en una imagen



Lo que necesitas ANTES

- Regresión logística y odds (Clases 20–21 y 31) y la matriz de confusión (Clases 30–31)
- Train / prueba, validación cruzada y fuga de información (Clase 30): aquí se usan en serio
- Estandarizar (z = (x − media)/sd) y el uso de un Pipeline (Clase 31)
- Idea de regularización (Ridge y Lasso, Clases 30–31): C de la SVM juega un papel parecido

Ruta de la clase y datos

- **Clasificación supervisada → métricas → validación y fuga → regresión logística → SVM: margen, C, kernel, gamma → comparar con prueba → AUC**
- Datos: **Breast Cancer Wisconsin**, 569 muestras (357 benignas, 212 malignas), cuatro predictores (radio, textura, suavidad, concavidad) y la partición ya hecha: **398 de entrenamiento y 171 de prueba**. Clase positiva: maligno (1)
- Pregunta guía: ¿qué regla de decisión usar, con qué hiperparámetros y cómo sabré que funciona en casos nuevos?

Puentes con las clases vecinas

C31 → C37: la logística y las métricas de la Clase 31 aquí se comparan con la SVM en una misma partición. **C34–C36 → C37**: árboles, bosques y boosting son otros clasificadores de la «familia de 15 métodos» que menciona el profesor; no se repiten aquí. **C37 → C38**: la evaluación de modelos supervisados (Eloy Alvarado) profundiza AUC, umbrales y validación; aquí solo se usa lo necesario para comparar tres modelos.

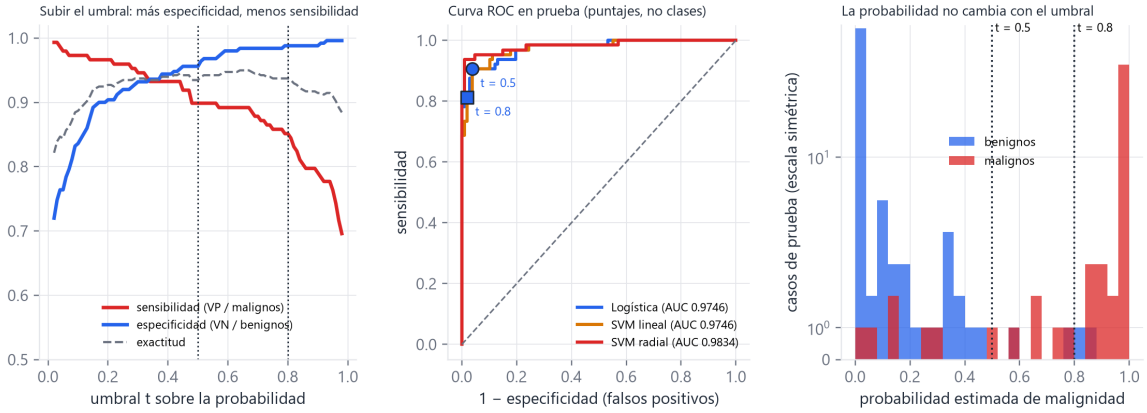
Clase 37 · Regresión logística: de la probabilidad a la clase

Fuente: transcripción de la clase + diapositivas + scripts clasificadores.py / .R + wisconsin_clase.csv del profesor Pedro Luiz Ramos

La regresión logística modela directamente la probabilidad de la clase positiva con un predictor lineal $\eta = \beta_0 + \beta \cdot x$ y la función logística. El umbral convierte la probabilidad en clase; cambiarlo mueve la decisión, no la probabilidad.

Conceptos clave

1. **Puntaje, probabilidad y odds.** $p = 1/(1 + e^{-\eta})$; $\eta = 0$ equivale a $p = 0.5$. Cada unidad de x_j multiplica los *odds* $p/(1 - p)$ por e^{β_j} (asociación condicional, no efecto causal).
2. **Umbral t.** Clase 1 si $p \geq t$. Con $t = 0.5$ es lo habitual, pero se puede fijar con un criterio previo (costos, validación cruzada), nunca mirando la prueba.
3. **Sensibilidad y especificidad.** Sensibilidad = VP/(VP + FN) (malignos detectados); especificidad = VN/(VN + FP) (benignos bien clasificados). Subir t gana especificidad y pierde sensibilidad.
4. **R frente a Python.** `glm(family = binomial)` no penaliza; `LogisticRegression()` **sí** penaliza por defecto: hay que poner `penalty=None`.



Izquierda: sensibilidad, especificidad y exactitud según el umbral (validación cruzada en entrenamiento; líneas en 0.5 y 0.8). Centro: curvas ROC en prueba (los puntos son la logística con t = 0.5 y 0.8). Derecha: la probabilidad de cada caso de prueba no cambia con el umbral; cambia a cuántos se deja de un lado.

Fórmulas y reglas

$p = 1 / (1 + e^{-\eta})$, $\eta = \beta_0 + \beta \cdot x$

$\text{odds} = p / (1 - p) = e^{\eta}$

odds nuevos / odds antiguos $= e^{\beta_j}$

frontera con $t = 0.5$: $\beta_0 + \beta \cdot x = 0$ (un hiperplano)

En Python

```
LogisticRegression(penalty=None,
                   max_iter=2000)
p = modelo.predict_proba(X)[:, 1]
clase = (p >= 0.5).astype(int)
```

Con los datos de la clase

- Prueba de 171 casos con t = 0.5: VN 103, FP 4, FN 6, VP 58 → exactitud 94.15 %, sensibilidad 90.62 %, especificidad 96.26 %
- Con t = 0.8: VN 105, FP 2, FN 12, VP 52 → 14 errores en vez de 10 (como se vio en clase); sensibilidad 81.25 %, especificidad 98.13 %
- Coeficientes de R: intercepto −42.2511, radio 1.3399, textura 0.4421 ($e^{0.4421} = 1.56$), suavidad 124.9794, concavidad 21.4077; caso nuevo (14, 20, 0.10, 0.08): $p = 0.3919$ → clase 0

Cuidado
Coeficientes de Python (estandarizados) y de R (originales) difieren en escala pero dan las **mismas probabilidades**. «+1» de suavidad (rango 0.05–0.16) no tiene sentido; mejor «+0.01»: odds × 3.49.

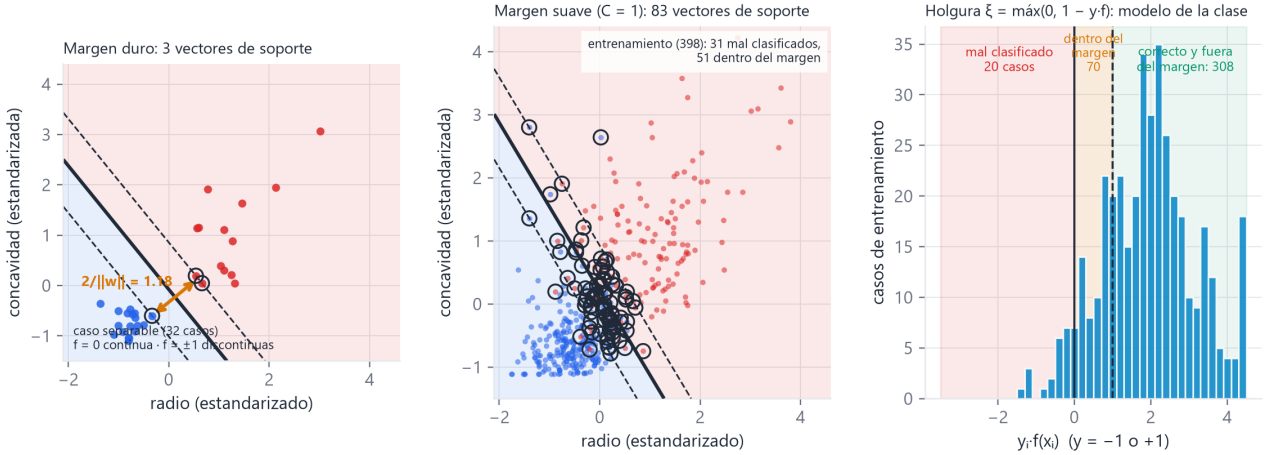
Clase 37 · SVM: hiperplano, margen y vectores de soporte

Fuente: transcripción de la clase + diapositivas + scripts clasificadores.py / .R + wisconsin_clase.csv del profesor Pedro Luiz Ramos

La SVM busca la frontera f(x) = w·x + b = 0 que separa las clases con el mayor margen. El signo de f(x) decide la clase y solo los vectores de soporte (los casos sobre el margen o dentro de él) determinan la solución.

Conceptos clave

1. **Hiperplano y puntaje.** f(x) = w·x + b. w da la orientación, b desplaza la frontera. La distancia geométrica de un punto a la frontera es $|f(x)|/\|w\|$.
2. **Margen máximo.** Con $y_i\, f(x_i) \geq 1$ el ancho total del margen es $2/\|w\|$; minimizar $\|w\|^2$ lo maximiza. Los puntos con $y_i\, f(x_i) = 1$ son los vectores de soporte.
3. **Margen suave.** Si las clases se superponen se permite una holgura $\xi_i = \max(0, 1 - y_i\, f(x_i))$: 0 si el caso está bien y fuera del margen; entre 0 y 1 si está dentro; mayor que 1 si está mal clasificado.
4. **Etiquetas −1 y +1.** Son una convención para explicar la geometría; en SVC y `svm()` las clases pueden seguir siendo 0 y 1.



Izquierda: margen duro en un subconjunto separable de 32 casos (3 vectores de soporte; ancho $2/\|w\| = 1.18$ en unidades estandarizadas). Centro: margen suave con C = 1 sobre los 398 casos de entrenamiento (radio y concavidad): 83 vectores de soporte. Derecha: holgura del modelo de la clase (4 variables, lineal, C = 0.1).

Fórmulas y reglas

$f(x) = w \cdot x + b$; frontera $f(x) = 0$

distancia $= |f(x)| / \|w\|$; ancho del margen $= 2 / \|w\|$

$\xi_i = \max(0, 1 - y_i\, f(x_i))$

mín $\frac{1}{2}\|w\|^2 + C \sum \xi_i$

En Python

```
svm = SVC(kernel='linear', C=0.1)
svm.fit(X_train_esc, y_train)
svm.decision_function(X) # f(x)
svm.support_vectors_, svm.n_support_
```

Con los datos de la clase

- Modelo de la clase (lineal, C = 0.1, 4 variables): 93 vectores de soporte (47 + 46); $\|w\| = 1.73$
- Entrenamiento (398): 20 mal clasificados, 70 dentro del margen y 308 correctos y fuera del margen
- Matriz en prueba: VN 103, FP 4, FN 10, VP 54 → 14 errores; exactitud 91.81 %, sensibilidad 84.38 %, especificidad 96.26 %
- Cuenta de la diapositiva: w = (1, −1), b = −0.5, x = (2, 1) → f = 0.5, $\|w\| = \sqrt{2}$, distancia 0.354

Cuidado
Un margen amplio **no garantiza** por sí solo buen desempeño en datos nuevos. Y f(x) **no es una probabilidad**: sus valores no están entre 0 y 1.

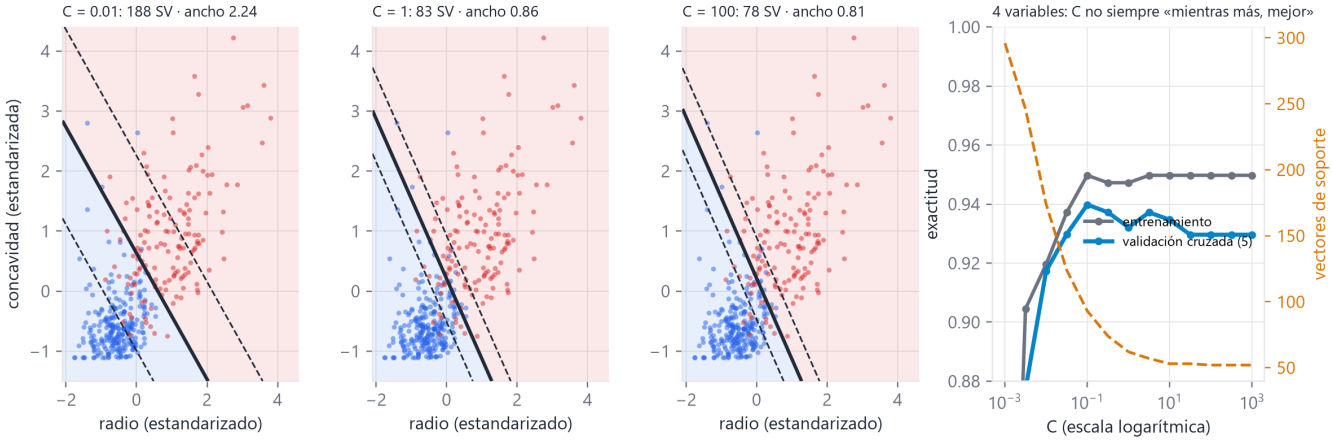
Clase 37 · Margen suave y el hiperparámetro C

Fuente: transcripción de la clase + diapositivas + scripts clasificadores.py / .R + wisconsin_clase.csv del profesor Pedro Luiz Ramos

C es el peso de las violaciones del margen en la función que se minimiza. C pequeño: se toleran más violaciones, margen ancho y regularización más fuerte; C grande: se castigan casi todas, margen estrecho y frontera más pegada al entrenamiento. No existe «el mejor C» sin validación.

Conceptos clave

1. **Qué controla.** En mín ½‖w‖² + C Σ ξᵢ, C mide cuánto pesa Σ ξᵢ frente a ‖w‖². Con Σ ξᵢ = 4: contribución 0.4 si C = 0.1 y 40 si C = 10.
2. **Se ve en el margen y en los soportes.** Al subir C el ancho 2/‖w‖ baja y quedan menos vectores de soporte (menos casos violan el margen).
3. **Tampoco «mientras más, mejor».** Con C muy pequeño hay subajuste; con C muy grande la frontera se ajusta a particularidades del entrenamiento. La validación cruzada decide.
4. **C de Python = cost de R.** Ambos son el mismo parámetro de libsvm; por eso R y Python entregan las mismas matrices de confusión con los mismos datos y escala.



Tres valores de C con un kernel lineal sobre radio y concavidad: el margen se estrecha de 2.24 a 0.86 y 0.81 y los soportes bajan de 188 a 83 y 78. Derecha: con las 4 variables, la exactitud de validación cruzada sube hasta C ≈ 0.1 y luego se mantiene o baja ligeramente.

Fórmulas y reglas

objetivo = ½‖w‖² + C · Σ ξᵢ

C ↓ : margen ancho, más violaciones, más simple

C ↑ : margen estrecho, menos violaciones, más flexible

ancho = 2 / ‖w‖

En Python

```
SVC(kernel='linear', C=0.1)
# R: svm(..., kernel='linear', cost=0.1)
GridSearchCV(pipe, {'svc__C': [0.1, 1, 10]},
             cv=5)
```

Con los datos de la clase

- 2 variables: C = 0.01 → 188 soportes y ancho 2.24; C = 1 → 83 y 0.86; C = 100 → 78 y 0.81
- 4 variables: validación cruzada (5 folds) 0.917 con C = 0.01, 0.940 con C = 0.1 y 0.930 con C = 1000; mejor C ≈ 0.1
- Exactitud de prueba del lineal: 0.918 con C = 0.1 y 0.936 con C ≥ 0.3: la prueba no se usa para elegir C
- El profesor fija C = 0.1 (lineal) y C = 10 (radial) **antes** de ver la prueba

Cuidado
«Con C pequeño no dejo que acepte errores» es lo contrario: C **pequeño** tolera **más** violaciones. Y elegir C mirando la prueba convierte la prueba en validación.

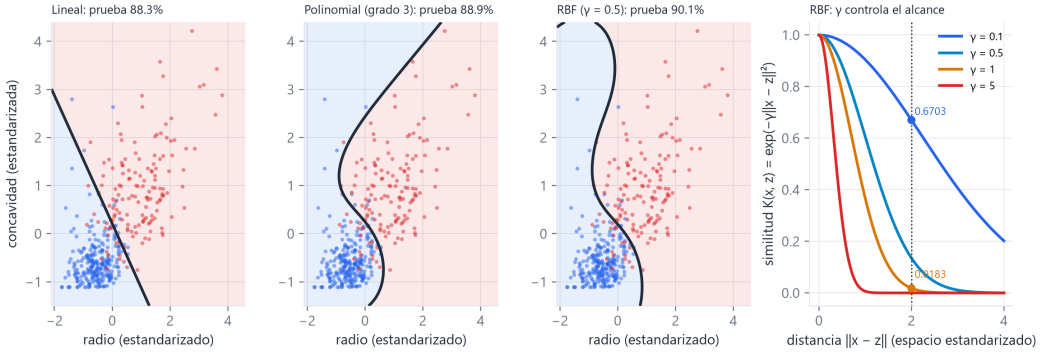
Clase 37 · Kernels y gamma: fronteras curvas

Fuente: transcripción de la clase + diapositivas + scripts clasificadores.py / .R + wisconsin_clase.csv del profesor Pedro Luiz Ramos

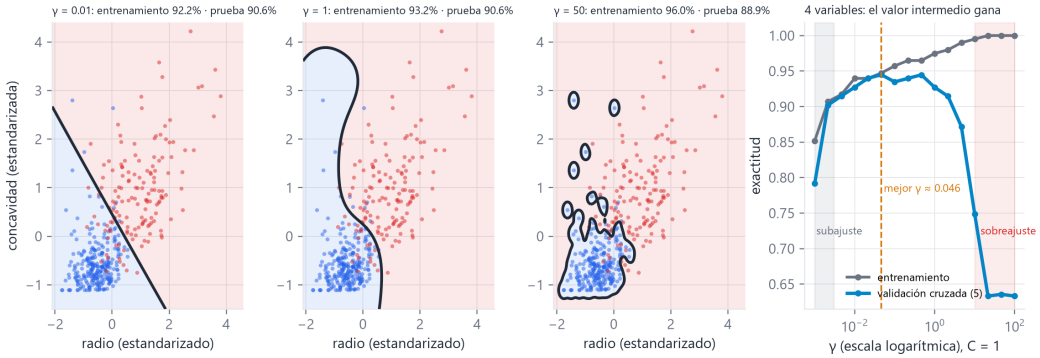
Un kernel K(x, z) = ⟨φ(x), φ(z)⟩ calcula un producto interno en un espacio de características sin construirlo (truco del kernel): la frontera es lineal allí y curva en el espacio original. En el kernel radial (RBF) el parámetro γ fija el alcance de la similitud.

Conceptos clave

1. **Tres kernels.** Lineal x·z; polinomial $(\gamma\, x\cdot z + r)^d$ (hiperparámetros γ, r y grado d); radial $\exp(-\gamma\|x - z\|^2)$. El lineal no tiene γ; el polinomial sí.
2. **γ pequeño y γ grande.** γ pequeño: la similitud decae lento y la frontera se parece a una recta (subajuste). γ grande: influencia muy local y fronteras irregulares que memorizan (sobreajuste).
3. **γ y C juntos.** Con γ muy grande ningún C salva el modelo; con γ muy pequeño el RBF se parece al lineal. Se buscan en rejilla (GridSearchCV, tune.svm). No hay un kernel mejor en general.



Lineal, polinomial (grado 3) y RBF sobre radio y concavidad (exactitud de prueba 88.3 %, 88.9 % y 90.1 %). Derecha: la similitud RBF cae con la distancia; a distancia 2, γ = 0.1 da 0.6703 y γ = 1 da 0.0183.



RBF con C = 1: γ = 0.01 (casi recta), 1 (curva suave) y 50 (islas alrededor de casos aislados). Derecha: con 4 variables la exactitud de entrenamiento sube siempre, la de validación cruzada tiene un máximo y cae.

Fórmulas y reglas

$K(x, z) = \exp(-\gamma\, \|x - z\|^2)$

$\|x - z\|^2 = 4$: γ = 0.1 → 0.6703; γ = 1 → 0.0183

$K_{poli} = (\gamma\, x\cdot z + r)^d$

En Python

```
SVC(kernel='rbf', C=10, gamma=0.1)
```

Con los datos de la clase

- 2 variables (C = 1): γ = 0.01 → entrenamiento 92.2 %, prueba 90.6 %; γ = 1 → 93.2 % / 90.6 %; γ = 50 → 96.0 % / 88.9 %
- 4 variables (C = 1): validación cruzada máxima 0.945 con γ ≈ 0.046; con γ = 100, 0.633 (entrenamiento 100 %)

Cuidado
«Gamma entre 0.1 y 0.5 es lo mejor» no es una regla: depende de la escala de los datos (por eso se estandariza) y se decide con validación.

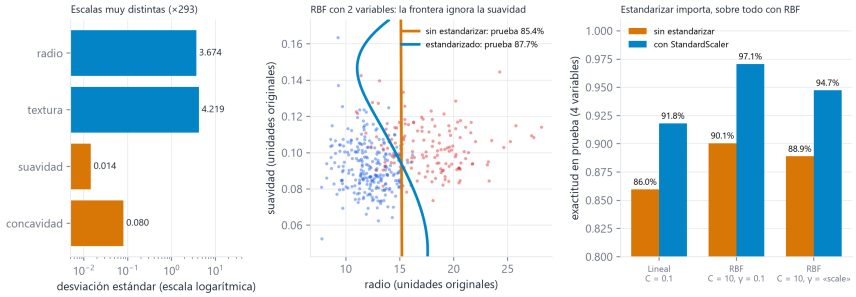
Clase 37 · Estandarizar, validar y no filtrar información

Fuente: transcripción de la clase + diapositivas + scripts clasificadores.py / .R + wisconsin_clase.csv del profesor Pedro Luiz Ramos

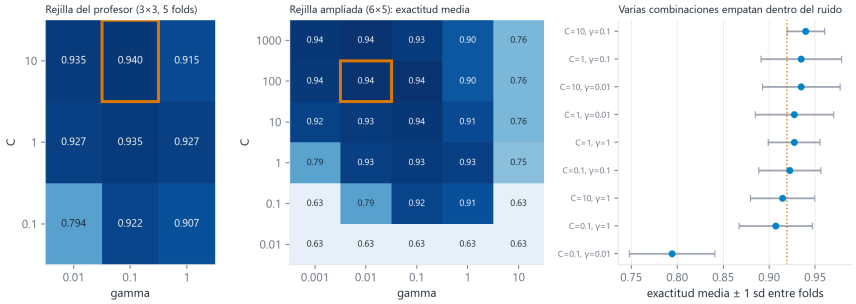
La SVM usa distancias: si una variable tiene una escala mucho mayor, domina. Se estandariza con la media y la desviación del entrenamiento y se dejan en un `Pipeline` para que dentro de cada fold se aprendan solo con los folds de ajuste. Los hiperparámetros se eligen en entrenamiento; la prueba se mira una vez.

Conceptos clave

1. **Tres funciones de los datos.** Entrenamiento: estima coeficientes, escalas y vectores de soporte. Validación: compara hiperparámetros y reglas de decisión. Prueba: evalúa el procedimiento ya elegido.
2. **Validación cruzada de k folds.** Se divide el entrenamiento en k partes; cada una valida una vez con un modelo ajustado en las otras k − 1. Con 3 valores de C, 3 de γ y 5 folds son 45 ajustes.
3. **Fuga de información.** Calcular media y sd (o imputar, o seleccionar variables) con toda la base antes de dividir contamina la validación. Tampoco se elige C, γ o t mirando la prueba.



Izquierda: las desviaciones estándar van de 0.014 (suavidad) a 4.2 (textura), ×293. Centro: RBF con radio y suavidad: sin estandarizar la frontera es casi una recta vertical en el radio. Derecha: exactitud de prueba con 4 variables, sin y con StandardScaler.



Izquierda: la rejilla 3×3 del profesor (mejor C = 10, γ = 0.1, exactitud media 0.9396). Centro: rejilla ampliada 6×5 (mejor C = 100, γ = 0.01, 0.9447). Derecha: las nueve combinaciones con ± 1 sd entre folds: varias empatan dentro del ruido.

Fórmulas y reglas

$$z = (x - \text{media}_{entr}) / \text{sd}_{entr}$$

$$\text{exactitud} = 1 - \text{error}$$

$$\text{ajustes} = |C| \cdot |\gamma| \cdot \text{folds}$$

En Python

```
make_pipeline(StandardScaler(), SVC())
GridSearchCV(modelo, {'svc__C': [...],
   'svc__gamma': [...]}, cv=5)
```

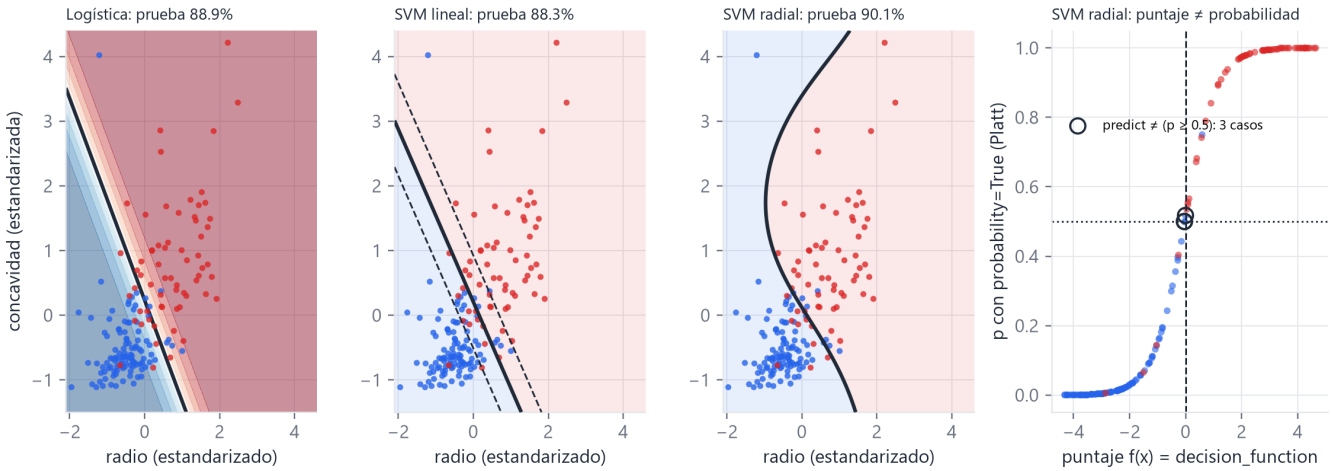
Con los datos de la clase

- Sin estandarizar la prueba cae de 91.8 % a 86.0 % (lineal) y de 97.1 % a 90.1 % (RBF)
- Rejilla de la diapositiva 62: mejor C = 10, γ = 0.1; la ampliada elige C = 100, γ = 0.01 (prueba 94.7 %, menos que el 97.1 % del profesor)

Cuidado
Una diferencia de 1 a 3 puntos **no es** evidencia de superioridad: en 171 casos, uno vale 0.58 puntos y los folds varían ± 2 a 4 puntos.

Comparar logística y SVM: fronteras, probabilidad y R ↔ Python

Primero el problema y el costo de los errores, luego el modelo



Con radio y concavidad: logística (probabilidad y frontera lineal), SVM lineal (recta con margen) y SVM radial (frontera curva), en prueba (88.9 %, 88.3 % y 90.1 %). Derecha: con `probability=True` la SVM calcula una probabilidad por ajuste de Platt a partir del puntaje; en 3 casos de prueba `predict` y «p ≥ 0.5» discrepan.

Modelo (prueba, 171 casos)	VN · FP · FN · VP	Exactitud	Sensib.	Especif.	Precisión	AUC
Logística (t = 0.5)	103 · 4 · 6 · 58	0.9415	0.9062	0.9626	0.9355	0.9746
SVM lineal (C = 0.1)	103 · 4 · 10 · 54	0.9181	0.8438	0.9626	0.9310	0.9746
SVM radial (C = 10, γ = 0.1)	106 · 1 · 4 · 60	0.9708	0.9375	0.9907	0.9836	0.9834
Línea base «siempre benigno»	107 · 0 · 64 · 0	0.6257	0.0000	1.0000	—	0.5

Criterio	Regresión logística	SVM lineal	SVM radial
Salida	probabilidad p(x)	puntaje f(x)	puntaje f(x)
Frontera aquí	lineal	lineal	no lineal
Interpretación	coeficientes y odds	pesos y margen	más difícil
Ajustes	penalización (opcional)	C y escala	C, γ y escala
R ↔ Python	glm(family = binomial) ↔ LogisticRegression(penalty=None)	svm(kernel = «linear», cost) ↔ SVC(kernel=«linear», C)	svm(kernel = «radial», cost, gamma) ↔ SVC(kernel=«rbf», C, gamma)

	Checklist del flujo	Cómo sé que lo hice bien
☐	Fijo la clase positiva (maligno = 1) y el orden de la matriz	filas = real, columnas = predicho, orden 0, 1
☐	Divido antes de estandarizar, imputar o ajustar	todo lo que aprende de los datos va en un Pipeline
☐	Elijo C, γ y el umbral con validación cruzada en entrenamiento	la prueba se mira una sola vez
☐	Comparo con la línea base y mido sensibilidad además de exactitud	los errores (FN y FP) casi nunca cuestan lo mismo
☐	Uso puntajes (no clases 0/1) para la AUC	decision_function o predict_proba
☐	Documento las limitaciones	base histórica de una región: no es una herramienta clínica

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales y qué conviene precisar (verificado con código)

Lo dicho en clase

- **«Con la base de 357 vamos a dividir en 70 y 30»**: 357 son solo los benignos; la base tiene 569 casos y la partición es 398 / 171 (250 + 148 y 107 + 64). La grabación dice además «559 muestras».
- **Regla «siempre uno» con 107 aciertos**: la regla que acierta 107 de 171 es la que predice **benigno (0)**, no 1. Siempre 1 acertaría 64 (37.4 %). La diapositiva 7 lo dice bien (0.626).
- **«Para SVM hay que usar −1 y 1»**: es solo la convención para la geometría; SVC y `svm()` aceptan 0/1 y dan las mismas predicciones (los scripts usan 0 y 1; lo aclara la diapositiva 31).
- **«El estimador de máxima verosimilitud tiene garantizado que la solución es única»**: no siempre. Con **separación perfecta** la verosimilitud sigue creciendo al aumentar los coeficientes y no hay máximo finito; con colinealidad perfecta tampoco es única (se ve en el notebook).
- **Falsos negativos**: «la persona va a ser mal pagador y predecimos buen pagador» es un falso **positivo** con la codificación 1 = buen pagador. Un falso negativo es lo contrario (real 1, predicho 0).
- **Kernels e hiperparámetros**: «el polinomial tiene un hiperparámetro, el radial gamma, el lineal ninguno». El polinomial $(\gamma\, x{\cdot}z + r)^d$ tiene tres: γ, r y el grado d; solo el lineal no tiene γ.
- **«Cuando voy a hacer inferencia, siempre me voy con regularización»**: la penalización **sesga** los coeficientes y no entrega los intervalos de confianza clásicos; es una herramienta de predicción y selección. Para inferir hay que tener cuidado (inferencia tras selección).
- **«C = 1 y gamma = 0.1 … los dos valores iguales a 10»** (Python): el script fija C = 10 y γ = 0.1 (diapositiva 43); la frase de la grabación está mal transcrita o es un error al hablar.

Diapositivas y scripts

- **Diapositiva 28**: la interpretación «una unidad de suavidad multiplica los odds por $e^{124.98}$» no tiene sentido práctico: el rango de suavidad es 0.05 a 0.16. Mejor «+0.01» (×3.49) o usar variables estandarizadas. Los coeficientes de R (−42.2511, 1.3399, 0.4421, 124.9794, 21.4077) **se reproducen exactamente** con `LogisticRegression(penalty=None)` sin estandarizar (p = 0.3919 para el caso nuevo).
- **Diapositiva 63 (R: C = 1, γ = 0.1; Python: C = 10, γ = 0.1)**: no es una contradicción; en Python (10, 0.1) gana con 0.9396 y (1, 0.1) queda segunda con 0.9347, dentro de 1 sd. Y la rejilla ampliada elige otra (100, 0.01) con prueba 94.7 % frente al 97.1 % de la configuración del profesor: ningún número de prueba justifica elegir.
- **AUC**: la logística y la SVM lineal dan **la misma AUC** (0.9746) con distinta exactitud; la SVM radial 0.9834. La AUC mide el orden de los puntajes, no el punto de corte (diapositiva 64).
- **Probabilidad de la SVM**: `decision_function` no es probabilidad; `SVC(probability=True)` la calibra (Platt, con validación cruzada interna) y puede discrepar de `predict`: 3 casos y exactitud 95.3 % frente a 97.1 %.
- **Diapositiva 16**: escribe «p = 0,731» con coma y en el resto del material con punto; es solo formato. **Script Python**: `os.chdir(r"D:\codigos")` y `setwd("D:/codigos")` son rutas del profesor; hay que cambiarlas.

Transcripción con ruido

Whisper confundió Wisconsin («Luis Colzin»), glm / glmnet («RLM», «relíem net»), e1071 («e de 71»), Tibshirani, Breiman («Breeman»), k-folds («CAFOD»), Leave-One-Out, kernel («crânio», «k-neu»), ROC («rock», «roca»), chance («iance», «yance»), ChatGPT («chate PT»). El volumen cayó ≈ 12 dB desde ≈ 65 min (se transcribió una copia amplificada); el recreo (≈ 1:26:14) no está en el video. Frases dudosas de la clase se dejaron como el profesor las dijo.

Criterio de dominio para pasar a la siguiente clase

- ☐ Calculo a mano la probabilidad logística, los odds y la clase para un umbral dado
- ☐ Calculo exactitud, sensibilidad, especificidad y precisión desde una matriz de confusión y la comparo con la línea base
- ☐ Calculo f(x), la distancia a la frontera y el ancho del margen 2/||w||, e identifico vectores de soporte
- ☐ Explico el efecto de subir C o γ y por qué se eligen con validación cruzada en entrenamiento
- ☐ Resuelvo los bloques 1 a 7 del cuaderno con al menos 80 % de aciertos